Section 3 — Planning and Reasoning

How Agents Think Before They Act

Author: Suresh D R | AI Product Developer & Technology Mentor

---

Table of Contents

#	Topic
3.1	Planning Inside Agents
3.2	Chain-of-Thought — How It Works and How Agent Consumes It
3.3	Tree-of-Thoughts — How It Works and How Agent Consumes It
3.4	Planning Algorithms — A* and MCTS
3.5	Self-Consistency
3.6	Which Technique to Use — Decision Guide
3.7	Section Summary

What This Section Is About

Section 2 taught you how an agent runs — the loop, tools, ReAct, Reflexion. You saw the agent act step by step.

Section 3 goes one level deeper. It answers the question everyone wonders about:

How does the agent actually think before it acts?

When a task is simple — "What is 2 + 2?" — no thinking needed. Just answer. But when a task is complex — "Write a complete market research report on the Indian EV industry" — the agent cannot just start randomly. It needs to figure out the right approach first. That figuring out is what this section covers.

By the end you will understand five reasoning techniques — from the simplest (Chain-of-Thought) to the most advanced (MCTS). You will also understand exactly **how the agent receives and uses** each technique internally.

Why Reasoning Matters — The Quick Example

Look at the same task handled two different ways:

Task: A shopkeeper buys mangoes at Rs 20 and sells at Rs 35. Sells 150 mangoes. What is total profit and profit percentage?

Without reasoning:
The agent jumps to: "Profit is Rs 2250, percentage is 75%." Sometimes right. Sometimes wrong. No way to verify.

With Chain-of-Thought:

```
Cost price per mango    = Rs 20
Selling price per mango = Rs 35
Profit per mango        = 35 − 20 = Rs 15
Total profit            = 15 × 150 = Rs 2,250
Profit percentage       = (15 / 20) × 100 = 75%
```

Same answer — but every step is visible. If any step is wrong you can see exactly where. And on harder problems, writing the steps prevents errors from happening at all.

This is the core insight of Section 3: **structured thinking produces better answers.**

3.1 Planning Inside Agents

> **Analogy:** Before a road trip you look at the map, decide the route, identify fuel stops, and estimate time for each stretch. You do not just start driving and figure it out as you go. A plan gives direction. Without it you double back, miss turns, and run out of petrol.

When Does an Agent Need a Plan?

Task Type	Example	Plan Needed?
Simple, 1 step	"What is the capital of France?"	No — just answer
Simple maths	"Calculate 23 × 47"	No — just calculate
Medium, 3-5 steps	"Find the CEO of Infosys and their background"	Optional
Complex, 6+ steps	"Write a competitor analysis for Zepto"	Yes — plan first
Very complex	"Create a 4-week Python learning curriculum"	Definitely yes

The rule is simple: if missing one step would break the whole output, plan first.

Plan-and-Execute — Make the Full Plan Before Acting

The agent splits the task into two separate phases.

Phase 1 — Planning: Write the full step-by-step plan before touching any tool.

Phase 2 — Execution: Run each step in the plan one by one.

Example — EV Market Research Report

```
GOAL: "Write a market research report on the Indian EV industry
       with market size, top players, recent news, and investment outlook"

═══════════════════════════════════════════
PHASE 1 — PLAN (created before any action)
═══════════════════════════════════════════

Step 1:  Search for Indian EV market size and growth rate 2024
Step 2:  Find top 5 EV companies in India by sales volume
Step 3:  Research each company — products, pricing, market share
Step 4:  Search for latest EV news in India (past 3 months)
Step 5:  Search for government EV policies and subsidies
Step 6:  Find analyst reports on EV investment outlook India
Step 7:  Write executive summary
```

```
Step 8:  Write market size section using data from Step 1
Step 9:  Write competitive landscape using Steps 2 and 3
Step 10: Write news and policy section using Steps 4 and 5
Step 11: Write investment outlook using Step 6
Step 12: Format and add section headings

═══════════════════════════════════════════
PHASE 2 — EXECUTION (run each step in order)
═══════════════════════════════════════════

Run Step 1 → gets market size data
Run Step 2 → gets company names
Run Step 3 → gets company details
... and so on
```

Without a plan: agent searches randomly, writes bits out of order, forgets sections, repeats things. Output is messy.

With a plan: every step has a purpose. Nothing is forgotten. Sections appear in the right order because the order was decided before execution started.

Dynamic Replanning — Updating the Plan When Things Fail

Steps fail in the real world. Websites go down. Searches return nothing. Dynamic replanning means the agent updates its plan instead of crashing or skipping.

```
ORIGINAL PLAN:
  Step 3: Read Company X profile from their website

EXECUTION:
  Action:      read_url("companyX.com/profile")
  Observation: "404 Error — page not found"

WITHOUT replanning:
  Agent marks step 3 failed → skips it → continues
  Final report has a gap where Company X should be

WITH dynamic replanning:
  Agent thinks: "Website is down. I will search for
                 Company X in news articles instead."
```

```
  Updates plan:
    Step 3a: web_search("Company X India EV 2024")
    Step 3b: read top 2 news results about Company X
  Runs updated steps → gets the information needed
  Final report covers Company X — no gap
```

This is what separates a brittle agent (breaks on one failure) from a robust agent (adapts and finds another way).

3.2 Chain-of-Thought (CoT)

> **Analogy:** A doctor diagnosing a patient does not just look and say "you have X." They reason out loud — "fever + rash + recent travel suggests A or B. Blood test rules out A. Therefore B." Writing every step of reasoning is what makes the diagnosis reliable. CoT does the same for agents.

What Is Chain-of-Thought?

Instead of jumping to an answer, the agent writes out each step of thinking. The chain of thoughts IS the reasoning — each thought connects to the next.

The most remarkable thing: you do not change any code. You just add three words to the prompt: **"think step by step."**

```
WITHOUT CoT:
  Question: "A train leaves Mumbai at 9 AM at 80 km/h.
             Another leaves Pune (150 km away) at 10 AM
             at 100 km/h toward Mumbai. When do they meet?"
  Answer: "10:30 AM"  ← might be wrong, no working shown

WITH CoT ("think step by step"):
  "By 10 AM, Train A has travelled: 80 × 1 = 80 km
   Remaining gap = 150 − 80 = 70 km
   Combined speed = 80 + 100 = 180 km/h
   Time to close gap = 70 / 180 = 0.389 hours = 23 minutes
   They meet at 10:23 AM"
  Answer: "10:23 AM"  ← correct, working visible
```

The answer improved simply by asking the agent to think out loud.

How the Agent Actually Consumes CoT — What Happens Inside

This is the most important part. How does the agent actually use CoT? Where does it go? What does the LLM see?

CoT is delivered to the agent through the **system prompt or the user prompt**. The LLM reads it as part of its input. That is all. There is no special code or framework for CoT — it is just text in the prompt.

Here is exactly what the LLM receives when CoT is used:

```
═══════════════════════════════════════════════
WHAT THE LLM SEES INSIDE (the full prompt)
═══════════════════════════════════════════════

System message:
  "You are a helpful assistant.
   Always think step by step before giving your final answer.
   Show every step of your reasoning."

User message:
  "A shopkeeper buys at Rs 20, sells at Rs 35.
   Sells 150 items. What is profit and profit percentage?"

═══════════════════════════════════════════════
WHAT THE LLM PRODUCES (its output)
═══════════════════════════════════════════════

"Let me think step by step.

 Step 1: Profit per item = 35 − 20 = Rs 15
 Step 2: Total profit = 15 × 150 = Rs 2,250
 Step 3: Profit % = (15 / 20) × 100 = 75%

 Final Answer: Total profit is Rs 2,250. Profit percentage is 75%."
```

The CoT instruction is just text. The LLM reads it and formats its output accordingly. The "thinking" is part of the LLM's text generation — it produces the reasoning steps token by token the same way it produces any other text.

In a ReAct agent, CoT happens inside the Thought step:

```
═══════════════════════════════════════════════
HOW CoT FITS INTO THE ReAct AGENT LOOP
═══════════════════════════════════════════════

The ReAct prompt includes this instruction:
  "Always reason step by step in your Thought."

Now the Thought becomes a Chain-of-Thought:

  Thought: "I need to find the gold price and calculate for 15 grams.
            First I will look up the price.
            Then I will multiply: price × 15.
            Let me start with the lookup."

  Action: knowledge_base
  Action Input: gold price India

  Observation: "1 gram = Rs 6,500"

  Thought: "I have the price: Rs 6,500 per gram.
            Now calculate: 6,500 × 15 = Rs 97,500.
            I have both numbers needed for the answer."

  Final Answer: "15 grams of gold in India costs Rs 97,500
                 (at Rs 6,500 per gram)."

The CoT is inside the Thought. No extra code.
Just the instruction "reason step by step" in the prompt.
```

Zero-Shot vs Few-Shot CoT

Zero-Shot CoT — no examples given, just the instruction:

```
Prompt: "Think step by step.
         If you invest Rs 10,000 at 8% compound interest for 3 years,
         how much will you have?"

Agent:
  "Year 1: 10,000 × 1.08 = 10,800
   Year 2: 10,800 × 1.08 = 11,664
   Year 3: 11,664 × 1.08 = 12,597.12
   Answer: Rs 12,597.12"
```

Few-Shot CoT — give one example of the reasoning style first, then ask the question:

```
Example given in prompt:
  Q: "Store has 50 items. Sells 30% on Monday, 20% of remaining on Tuesday. How many left?"
  A: "Monday: 50 × 30% = 15 sold. Remaining: 35.
      Tuesday: 35 × 20% = 7 sold. Remaining: 28.
      Answer: 28 items."

Now ask the real question:
  Q: "Factory makes 200 units/day. 5% defect rate. Each defect costs Rs 500 to fix.
      What is daily repair cost?"

Agent follows the same style automatically:
  "Defective units = 200 × 5% = 10
   Daily cost = 10 × 500 = Rs 5,000
   Answer: Rs 5,000 per day."
```

Few-shot is more reliable because the agent has seen the exact format you want.

When CoT Helps vs When It Does Not

Situation	Use CoT?	Why
Multi-step maths	Yes	Prevents arithmetic errors
Logical reasoning	Yes	Makes reasoning checkable

Situation	Use CoT?	Why
Planning a sequence	Yes	Shows which steps come first
Simple factual question	No	Overhead with no benefit
Creative writing	No	Reasoning does not improve creativity
One-word answer needed	No	CoT makes output longer unnecessarily

General rule: if a human would benefit from writing out the steps, the agent benefits from CoT. If a human would just answer directly, CoT adds no value.

3.3 Tree-of-Thoughts (ToT)

> **Analogy:** A chess grandmaster does not make the first move that looks good. They think several moves ahead — "if I go here, opponent might go there, then I respond this way..." They explore multiple lines simultaneously, evaluate which is most promising, and choose the best. ToT makes agents think like a chess player.

What Is Tree-of-Thoughts?

Chain-of-Thought follows one line of reasoning — thought → thought → thought → answer. Tree-of-Thoughts branches the reasoning into multiple paths simultaneously. The agent generates several approaches, evaluates each one, follows the most promising, and prunes the ones that lead nowhere.

```
Chain-of-Thought (single path):
  Goal → Thought 1 → Thought 2 → Thought 3 → Answer

Tree-of-Thoughts (multiple paths explored):

  Goal ─┬─ Path A: A1 → A2 → dead end ✗
        ├─ Path B: B1 → B2 → B3 → best answer ✓
        └─ Path C: C1 → abandoned early ✗

Agent evaluates all three.
Path B is most promising → follow it to the answer.
```

A Real Example — The Water Jug Problem

Problem: You have a 3-litre jug and a 5-litre jug. Unlimited water supply. Measure exactly 4 litres.

```
PATH A — Fill the 5-litre jug first:
  Step 1: Fill 5L jug fully          (5L in 5L, 0L in 3L)
  Step 2: Pour into 3L until full    (2L in 5L, 3L in 3L)
  Step 3: Empty 3L jug               (2L in 5L, 0L in 3L)
  Step 4: Pour 2L from 5L into 3L  (0L in 5L, 2L in 3L)
  Step 5: Fill 5L jug fully          (5L in 5L, 2L in 3L)
  Step 6: Pour into 3L until full    (4L in 5L, 3L in 3L)
  ✓ 4 litres achieved in 6 steps

PATH B — Fill the 3-litre jug first:
  Step 1: Fill 3L jug                (3L in 3L, 0L in 5L)
  Step 2: Pour into 5L jug           (0L in 3L, 3L in 5L)
  Step 3: Fill 3L jug again          (3L in 3L, 3L in 5L)
  Step 4: Pour into 5L until full    (1L in 3L, 5L in 5L)
  Step 5: Empty 5L jug              (1L in 3L, 0L in 5L)
  Step 6: Pour 1L into 5L           (0L in 3L, 1L in 5L)
  Step 7: Fill 3L jug                (3L in 3L, 1L in 5L)
  Step 8: Pour into 5L jug           (0L in 3L, 4L in 5L)
  ✓ 4 litres achieved in 8 steps

Agent evaluates both paths:
  Path A achieves goal in 6 steps
  Path B achieves goal in 8 steps
  Path A is more efficient → agent chooses Path A
```

Without ToT the agent would have followed one path and stopped. With ToT it found the better solution.

How the Agent Actually Consumes ToT — What Happens Inside

This is the key question. How does the agent use ToT? Where does it go in the system?

ToT is implemented through a **structured multi-call process**. Unlike CoT which is a single prompt change, ToT requires the agent to make multiple LLM calls — one to generate candidate thoughts, another to evaluate them, and then more calls to expand the best ones.

Here is exactly what happens step by step:

```
═══════════════════════════════════════════════
HOW ToT IS CONSUMED BY THE AGENT INTERNALLY
═══════════════════════════════════════════════

CALL 1 — Generate candidate thoughts (breadth first)

  Prompt sent to LLM:
    "Task: [the problem]
     Generate 3 different possible first steps to solve this.
     Each step should be a different approach.
     Output them as: Approach 1: ... Approach 2: ... Approach 3: ..."

  LLM returns:
    "Approach 1: Fill the 5L jug first and pour into 3L
     Approach 2: Fill the 3L jug first and pour into 5L
     Approach 3: Fill both jugs and combine"

═══════════════════════════════════════════════

CALL 2 — Evaluate each candidate

  Prompt sent to LLM:
    "Task: [the problem]
     Rate each approach on a scale of 1-10 for likelihood of success:
     Approach 1: Fill 5L jug first...
     Approach 2: Fill 3L jug first...
     Approach 3: Fill both and combine...
     Output: Approach X: score, reason"

  LLM returns:
    "Approach 1: 8/10 — straightforward, fewer steps likely
     Approach 2: 7/10 — valid but may take more steps
     Approach 3: 2/10 — combining does not help here"
```

```
═══════════════════════════════════════════

CALL 3 — Expand the best candidate (Approach 1)

  Prompt sent to LLM:
    "Task: [the problem]
     Starting from: Fill the 5L jug first.
     Think step by step and complete the solution from here."

  LLM returns:
    Full step-by-step solution for Approach 1

═══════════════════════════════════════════

CALL 4 (optional) — Expand second best candidate (Approach 2)
  If Approach 1 fails, system falls back to Approach 2

═══════════════════════════════════════════

Final step: Return the completed solution from the best path
```

The key difference from CoT:

```
CoT:
  1 LLM call
  Prompt says "think step by step"
  LLM generates one reasoning chain in its output
  Simple. Fast. Works for most tasks.

ToT:
  Multiple LLM calls (typically 3 to 5+)
  Call 1: generate candidate approaches
  Call 2: evaluate and score each approach
  Call 3+: expand the best approach(es)
  More expensive. More powerful. For complex tasks only.
```

In an agent framework, ToT is orchestrated by the agent controller — the piece of code (like LangChain or LangGraph) that manages multiple LLM calls and decides which path to follow based on the evaluation scores.

CoT vs ToT — The Practical Difference

	Chain-of-Thought	**Tree-of-Thoughts**
LLM calls	1	3 or more
Reasoning paths	1	Multiple explored simultaneously
Speed	Fast	Slower
Cost	Low	Higher
Best for	Most tasks	Tasks with multiple valid approaches
How consumed	Single prompt instruction	Multiple coordinated LLM calls
Implementation	3 words in prompt	Orchestration logic needed

Rule of thumb: Start with CoT. If the agent keeps missing better solutions or getting stuck on one approach, add ToT.

When to Use Tree-of-Thoughts

Use ToT when:

- There are multiple valid ways to solve the problem
- You want the best solution, not just any solution
- The task involves strategy, creativity, or planning with alternatives
- Going down the wrong path wastes significant effort

Do not use ToT when:

- The task is straightforward with one obvious approach
- Speed matters more than finding the optimal solution

- Cost of extra LLM calls is a concern

Examples where ToT is worth it:

- Writing — explore multiple story angles, pick the most compelling
- Debugging — explore multiple possible root causes of a bug
- Business strategy — explore multiple approaches, evaluate risk/reward
- Architecture design — explore multiple system designs, pick most scalable

---

3.4 Planning Algorithms — A* and MCTS

> **Analogy for A*:** Google Maps finding the fastest route. It does not try every possible road. It uses a straight-line distance estimate to decide which roads are worth exploring and which to skip. Fast, finds the optimal route.

> **Analogy for MCTS:** A cricket captain deciding batting strategy. Cannot calculate every possible future mathematically. Instead they simulate several scenarios mentally — "if I send this batsman first, based on what I have seen, this is likely to work out best." Monte Carlo means using simulation to guide decisions.

A* — Finding the Optimal Sequence of Steps

A* finds the most efficient path from a starting state to a goal state. It balances actual cost (steps already taken) with estimated remaining cost (how far to the goal).

```
The formula:

  f(n) = g(n) + h(n)

  g(n) = actual cost from start to current state n
         (how many steps taken, how expensive they were)

  h(n) = estimated cost from current state n to goal
         (the heuristic — an educated guess of remaining effort)

  f(n) = total estimated cost through this state
```

```
A* always explores the state with the lowest f(n) first.
This ensures it finds the optimal path.
```

A* Example in an Agent — Research Task Planning

```
GOAL: Compile a report on renewable energy in India

Current state: No information gathered

Possible next actions and their f(n) values:
  Action A: search "India solar capacity 2024"
            g = 1 step, h = 5 steps estimated remaining → f = 6
  Action B: search "India wind energy"
            g = 1 step, h = 6 steps estimated remaining → f = 7
  Action C: read 50-page government PDF in full
            g = 5 steps (expensive!), h = 4 remaining → f = 9

A* picks Action A (lowest f = 6)

After Action A returns solar data:
  Action D: search "India wind energy" (next logical step)
            g = 2, h = 4 remaining → f = 6
  Action E: search "India renewable investment 2024"
            g = 2, h = 3 remaining → f = 5

A* picks Action E (lowest f = 5)

A* keeps picking the most promising next action based on
both actual progress AND estimated remaining work.
Result: agent reaches the complete report by the most
efficient sequence of steps.
```

MCTS — When the Future Is Unpredictable

MCTS is used when the action space is too large or unpredictable for A*. It simulates many random future scenarios and uses those simulations to figure out which actions tend to lead to good outcomes.

The Four Phases of MCTS

```
Repeats many times until confident enough to act:

1. SELECTION
   From current state, move through the tree
   Always pick the child that balances:
     exploration (paths not tried much yet) and
     exploitation (paths that worked well before)

2. EXPANSION
   When you reach a state with unexplored options,
   add one unexplored option to the tree as a new node.
   This is a new possible action being considered.

3. SIMULATION (the Monte Carlo part)
   From the new node, play out a random sequence of actions
   all the way to the end (task complete or failed).
   This is a fast random "test run" of the future.

4. BACKPROPAGATION
   Take the result of the simulation (success or failure)
   and update the score of every node on the path back to root.
   Good outcomes increase the score of nodes on that path.
   Future selections will prefer higher-scored nodes.
```

MCTS Example — Agent Handling a Customer Complaint

```
Current state: Customer complains about delayed delivery

Four possible responses the agent can make:
  A: Apologise + offer 10% discount
  B: Explain delay + promise new timeline
  C: Escalate to supervisor
  D: Offer full refund immediately

MCTS simulates each option many times:
```

```
Option A simulations (apologise + discount):
  Sim 1: Customer accepts, leaves 4-star review
  Sim 2: Customer asks for bigger discount but stays
  Sim 3: Customer satisfied, no further complaint
  Average outcome score: 7.2 / 10

Option B simulations (explain + timeline):
  Sim 1: Customer not satisfied, escalates anyway
  Sim 2: Customer leaves 2-star review
  Sim 3: Customer waits but still unhappy
  Average outcome score: 3.1 / 10

Option C simulations (escalate):
  Sim 1: Supervisor resolves quickly — good
  Sim 2: Wait time for supervisor too long — bad
  Sim 3: Supervisor resolves with bigger discount — costly
  Average outcome score: 5.8 / 10

Option D simulations (full refund):
  Sim 1: Customer very happy — but company loses money
  Sim 2: Customer happy, leaves 5-star review — but costly
  Average outcome score: 6.5 / 10 (good outcome, high cost)

MCTS picks Option A — highest average score (7.2)
Agent responds: apologise and offer 10% discount.
```

A* vs MCTS — When to Use Which

	A*	**MCTS**
Best when	Action space is well-defined	Action space is large and unpredictable
Finds	The optimal path	The statistically best action
Speed	Fast (with good heuristic)	Slower (needs many simulations)
Guarantees optimal?	Yes	No — best estimate
Real-world use in agents	Navigation, structured tasks	Games, strategy, complex decisions

3.5 Self-Consistency

> **Analogy:** You ask three doctors the same question. If all three say the same thing, you are confident. If one disagrees, you investigate further. You are using majority opinion to increase confidence. Self-Consistency does the same — run reasoning multiple times, take the most common answer.

What Is Self-Consistency?

Instead of running reasoning once and trusting it, run it multiple times with slight variations and take the majority vote. The insight: random errors in different runs tend to disagree with each other. The correct answer tends to appear most consistently across runs.

```
Question: "Is 91 a prime number?"

WITHOUT Self-Consistency (one run):
  Run 1: "91 seems prime"  ← WRONG (91 = 7 × 13)
  You got unlucky. No way to know this is wrong.

WITH Self-Consistency (5 runs):
  Run 1: "91 = 7 × 13, NOT prime" ✓
  Run 2: "91 is divisible by 7, NOT prime" ✓
  Run 3: "91 seems prime" ✗  (the error, but minority)
  Run 4: "7 × 13 = 91, NOT prime" ✓
  Run 5: "NOT prime — factors are 7 and 13" ✓

  Majority vote: NOT prime (4 out of 5) ← correct
  Confidence: 80% (4/5 agreed)
```

The error was caught because it was in the minority. Truth is consistent — errors are inconsistent.

A Real Business Example

Task: Classify 1000 customer reviews as Positive, Negative, or Neutral. These results drive business decisions so accuracy matters.

```
Review: "The product is okay but delivery was absolutely terrible"

Single run:
  Result: "Negative"
  Correct? Maybe. No way to know confidence level.

Self-Consistency (5 runs):
  Run 1: "Negative — delivery complaint dominates"
  Run 2: "Negative — more negative than positive content"
  Run 3: "Neutral — product okay balances delivery complaint"
  Run 4: "Negative — customer clearly dissatisfied overall"
  Run 5: "Negative — strong negative signal from delivery"

  Result: Negative (4/5 agreed)
  Confidence: High → use this result

Another review: "Absolutely love it! Fast delivery and great quality!"
  All 5 runs: Positive
  Confidence: Very high (5/5 agreed) → no question

Another review: "It's fine I guess"
  Run 1: Neutral  Run 2: Neutral  Run 3: Positive
  Run 4: Neutral  Run 5: Negative
  Confidence: Low (only 3/5 agreed on Neutral)
  → Flag this one for human review
```

Self-Consistency gives you both the answer AND a confidence score. Low confidence = flag for human review. This is extremely valuable in production systems.

The Cost vs Reliability Tradeoff

```
Runs    Reliability       Cost
──────────────────────────────────
1 run   Base level        1x
3 runs  Good — catches most errors   3x
5 runs  High reliability  5x
7 runs  Very high         7x
```

```
Recommendation:
  Normal tasks       → 1 run (fast, cheap)
  Important outputs  → 3 runs (good balance)
  Critical decisions → 5 runs (worth the cost)
  Medical / legal    → 7 runs (reliability non-negotiable)
```

3.6 Which Technique to Use — Decision Guide

```
What kind of task is it?
         |
         ▼
Simple — 1 or 2 steps?
  Yes → Just use ReAct. No special reasoning needed.
  No  → continue below

         |
         ▼
Complex with many steps?
  Yes → Use Plan-and-Execute (make full plan first)
        Add Chain-of-Thought inside each step

         |
         ▼
Multiple valid approaches to explore?
  Yes → Use Tree-of-Thoughts
        (explore paths, evaluate, pick best)

         |
         ▼
Action space huge and unpredictable?
  Yes → Use MCTS
        (simulate many futures, pick best action)

         |
         ▼
```

```
Result is important and errors are costly?
  Yes → Use Self-Consistency
        (run 3-5 times, take majority vote)
        Combine with any technique above

         |
         ▼
Well-defined steps but need optimal sequence?
  Yes → Use A*
        (cost + heuristic drives step selection)
```

3.7 Section Summary

Technique	What It Does	How Agent Consumes It	Best For
Plan-and-Execute	Makes full plan before acting	Planner LLM call → executor calls	Complex 6+ step tasks
Chain-of-Thought	Writes step-by-step reasoning	Single prompt with "think step by step" — one LLM call	Maths, logic, any multi-step reasoning
Tree-of-Thoughts	Explores multiple reasoning paths	Multiple LLM calls — generate → evaluate → expand	Problems with multiple valid solutions
A*	Finds optimal action sequence	Mathematical cost function guides step selection	Well-defined action spaces
MCTS	Simulates futures to pick best action	Many simulation runs → backpropagation	Large unpredictable action spaces
Self-Consistency	Runs reasoning multiple times, votes	N identical LLM calls → majority vote	High-stakes decisions, classification

---

The one thing to remember from Section 3:

Agents are only as smart as the reasoning you give them. The same LLM with a bad prompt gives mediocre answers. The same LLM with Chain-of-Thought, a good plan, and Self-Consistency gives reliable, accurate, well-structured answers. The reasoning technique is a multiplier on the LLM's capability.
